

Wheel Mask Segmentation Handoff

Date: 2026-05-18

Purpose

This document describes what we currently have for wheel-mask generation and how another team member can fine-tune the segmentation model on local hardware.

There are two related but different tasks:

1. **Wheel mask generation:** find wheel pixels or wheel candidate masks on a car image.
2. **Wheel candidate filtering:** classify candidate crops as valid or invalid before using expensive VLM or generation calls.

This handoff is about the first task: wheel mask generation.

Current Assets

Local Pretrained Model

The local handoff package includes:

`models/yolo11n-seg.pt`

This is the Ultralytics YOLO11n segmentation checkpoint pretrained on COCO segmentation data. It is a good starting point for fine-tuning because it already contains general visual features.

Important limitation:

The COCO-pretrained model is not wheel-specific.

It should be fine-tuned on a wheel segmentation dataset before production use.

Local checksum:

SHA256: 55ed65c56c91713d23e8402371c6c49a6fd84f257f7dce452e8d70e41dcbe152

Official reference: Ultralytics lists `yolo11n-seg.pt` as a YOLO11 instance segmentation model.

Existing Inference Code

The repository already has an ONNX inference module:

`src/wheel_seg.py`

It expects an exported ONNX model at:

`models/yolo11n-seg.onnx`

or another path set by:

`WHEEL_SEG_MODEL_PATH=/path/to/model.onnx`

The public function is:

`detect_wheel_mask(image_bytes: bytes) -> bytes`

It returns a PNG mask with the same size as the input image:

- 255 means wheel pixels;
- 0 means background.

The class id is controlled by:

WHEEL_SEG_CLASS_ID=0

After fine-tuning, this value must match the class index of **wheel** in the training dataset YAML.

Roboflow Candidate Pipeline

We also have a Roboflow-based candidate generator:

```
scripts/roboflow_probe.py
scripts/roboflow_benchmark.py
```

The current useful Roboflow model id is:

wheels-tires-body/1

Example command for one image:

```
ROBOFLOW_API_KEY=... python scripts/roboflow_probe.py path/to/car.jpg \
  --model-id wheels-tires-body/1 \
  --classes wheel \
  --top-n 0 \
  --output-dir tmp/roboflow
```

This saves:

- raw Roboflow JSON;
- wheel mask;
- combined mask;
- overlay for visual inspection.

Recommended Training Dataset Format

Use YOLO segmentation format. A minimal dataset should look like this:

```
wheel-seg-dataset/
  data.yaml
  images/
    train/
    val/
    test/
  labels/
    train/
    val/
    test/
```

Example data.yaml:

```
path: /absolute/path/to/wheel-seg-dataset
train: images/train
val: images/val
```

```
test: images/test
names:
  0: wheel
```

Each label file must have the same base name as the image file.

For segmentation, every line in a label file contains:

```
class_id x1 y1 x2 y2 x3 y3 ...
```

Coordinates are normalized to $[0, 1]$. They represent polygon points of one instance mask.

Recommended Fine-Tuning Setup

Create a separate training environment on the machine with GPU:

```
python -m venv .venv-yolo
source .venv-yolo/bin/activate
python -m pip install --upgrade pip
pip install ultralytics
```

Check that GPU is visible:

```
yolo checks
```

Fine-tune from the pretrained model:

```
yolo segment train \
  model=models/yolo11n-seg.pt \
  data=/absolute/path/to/wheel-seg-dataset/data.yaml \
  imgsz=640 \
  epochs=80 \
  batch=16 \
  device=0 \
  project=runs/wheel-seg \
  name=yolo11n-wheel-seg
```

If GPU memory is limited, reduce batch size:

```
batch=8
```

If the dataset is small, start with fewer epochs:

```
epochs=30
```

Validation

After training, validate the best checkpoint:

```
yolo segment val \
  model=runs/wheel-seg/yolo11n-wheel-seg/weights/best.pt \
  data=/absolute/path/to/wheel-seg-dataset/data.yaml \
  imgsz=640 \
  device=0
```

Important metrics:

- mask mAP50;
- mask mAP50-95;
- precision;
- recall;
- visual quality of masks on hard examples.

For our product, recall is very important. If the model misses a real wheel, later pipeline stages cannot recover it.

Export for This Repository

The production-oriented repository code uses ONNX through `onnxruntime`, not PyTorch. After fine-tuning, export the best checkpoint:

```
yolo export \
  model=runs/wheel-seg/yolo11n-wheel-seg/weights/best.pt \
  format=onnx \
  imgsz=640
```

Copy the exported file into this repository:

```
cp runs/wheel-seg/yolo11n-wheel-seg/weights/best.onnx models/yolo11n-wheel-seg.onnx
```

Set environment variables:

```
export WHEEL_SEG_MODEL_PATH=models/yolo11n-wheel-seg.onnx
export WHEEL_SEG_CLASS_ID=0
```

If the class order is different, change `WHEEL_SEG_CLASS_ID`.

Smoke Test in This Repository

Install runtime dependencies:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
```

Run a small Python smoke test:

```
python - <<'PY'
from pathlib import Path
from src.wheel_seg import detect_wheel_mask

image_path = Path("webapp/cover.jpg")
mask_png = detect_wheel_mask(image_path.read_bytes())
Path("tmp/wheel-mask-smoke.png").parent.mkdir(parents=True, exist_ok=True)
Path("tmp/wheel-mask-smoke.png").write_bytes(mask_png)
print("saved tmp/wheel-mask-smoke.png")
PY
```

Open the output mask and check that wheel regions are white and the background is black.

Suggested Fine-Tuning Pipeline

1. Collect or export wheel segmentation images.
2. Keep source images outside Git if they are large.
3. Annotate wheel instance masks in Roboflow, CVAT, Label Studio, or another segmentation tool.
4. Export the dataset in YOLO segmentation format.
5. Train `yolo11n-seg.pt` on the wheel dataset.
6. Validate on a held-out test split.
7. Review masks on hard cases:
 - wet road reflections;
 - showroom floor reflections;
 - dark wheels;
 - partial or occluded wheels;
 - background vehicles;
 - low-light images.
8. Export the best model to ONNX.
9. Run the smoke test in this repository.
10. Share:
 - `best.pt`;
 - exported `.onnx`;
 - `data.yaml`;
 - validation metrics;
 - 10-20 overlay examples.

Recommended Dataset Size

Minimum useful target:

100-200 images with wheel masks

Better course or prototype target:

300-500 images

Pre-production target:

1000+ images with hard cases

Do not split by crop. Split by original source image or source video. This avoids train-test leakage.

Notes for Product Use

The segmentation model should maximize wheel recall. It is usually better to keep a few extra candidate masks and remove false positives later with the candidate filter or VLM.

Recommended product flow:

```
wheel segmentation model
-> all reasonable wheel candidate masks
-> lightweight candidate filter
-> VLM fallback for uncertain cases
-> final mask for generation
```

Avoid using `top_n=2` in hard cases. A small rear wheel can be dropped before later stages see it.

References

- Ultralytics YOLO11 documentation: <https://docs.ultralytics.com/models/yolo11/>
- Ultralytics segmentation task documentation: <https://docs.ultralytics.com/tasks/segment/>
- Roboflow dataset export documentation: <https://docs.roboflow.com/exporting-data>